

Vector Databases, *Explained* Without Embeddings

Five checks to run before you provision a vector store — count your data, test the WHERE clause, and catch retrieval that lies.

NAME

DATE

VECTOR COUNT

1

Count Your Vectors Before You Provision Anything

- Count docs/chunks you plan to embed. Under **~100k**? Skip Pinecone.
- Write the NumPy top-k below. Run it on *your* embedded data.
- Record real latency at your row count vs. what a hosted index bills monthly.

```
v = vectors / np.linalg.norm(vectors, axis=1, keepdims=True)
q = query / np.linalg.norm(query)
```

ROWS · LATENCY · \$/MO

2

Build the 'WHERE Clause' Test

- Write **3 real queries** your users would type.
- Each one — could a `WHERE` or `LIKE` answer it (order number, error code, date)?
- Or does it need *meaning* match (vague complaint, rephrased question)?
- Sort into two buckets. If it's all bucket one, you don't need embeddings yet.

SQL BUCKET · VECTOR BUCKET

3

Reproduce the Checkout/Cart Failure

- Pick **one ambiguous query** from your data — concepts that cluster but differ.
- Run it through retrieval. Pull the actual *top-5* chunks back.
- Read them by hand. One sentence: `about the same thing or the exact thing?`

SAME THING VS. EXACT THING

4

Add One Hybrid Filter

- Name **one hard constraint** — date, ID, status code, exact term.
- Add keyword/metadata filter on top of vector search for it.
- Run same query with and without filter. Note what changed.

FILTER · BEFORE/AFTER DIFF

5

Log and Read Your Retrieved Chunks

- Log the **raw chunks** behind every RAG answer, not just the answer.
- Pull your last *5 answers*. Read what actually fed each one.
- Flag any answer that sounds confident but chunks don't support it — the `Q3 migration` failure mode.

CONFIDENT BUT WRONG?